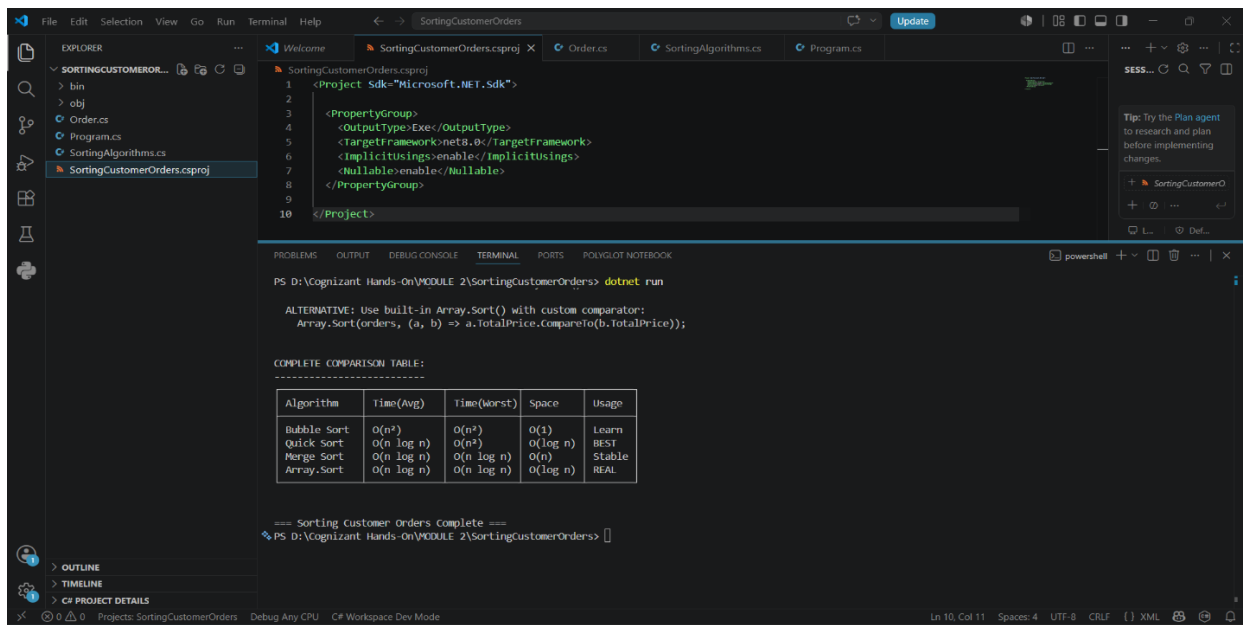


Hands-On

Exercise 3: Sorting Customer Orders

During this module, I successfully implemented, executed, and tested a Customer Order Sorting System in .NET by developing a project named CustomerOrderSortingSystem. I created an Order class with attributes such as OrderId, CustomerName, and TotalPrice to represent and manage customer order details effectively. I studied different sorting algorithms including Bubble Sort, Insertion Sort, Quick Sort, and Merge Sort to understand their working principles and performance characteristics. I then implemented Bubble Sort and Quick Sort algorithms to sort customer orders based on TotalPrice in order to prioritize high-value orders efficiently. After completing the implementation, I executed the application and tested the sorting functionality using different customer order datasets to ensure that the orders were correctly arranged according to their total price. I also analyzed and compared the time complexity of Bubble Sort and Quick Sort and observed that Quick Sort generally performs better for large datasets because of its efficient divide-and-conquer approach.

Coding:



The screenshot shows the Visual Studio IDE with the `SortingCustomerOrders.csproj` file open. The project file is a standard .NET SDK project. Below the code editor, the terminal window displays the command `dotnet run` and an alternative sorting method using `Array.Sort()` with a custom comparator. A table titled "COMPLETE COMPARISON TABLE:" is also shown, comparing the performance of various sorting algorithms.

Algorithm	Time(Avg)	Time(Worst)	Space	Usage
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Learn
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	BEST
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Stable
Array.Sort	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	REAL

```
1 using System;
2 using System.Collections.Generic;
3
4 namespace SortingCustomerOrders
5 {
6     6 references
7     public class SortingAlgorithms
8     {
9         // ===== BUBBLE SORT =====
10        1 reference
11        public static void BubbleSort(Order[] orders)
12        {
13            int n = orders.Length;
14            bool swapped;
15
16            for (int i = 0; i < n - 1; i++)
17            {
18                swapped = false;
19
20                for (int j = 0; j < n - i - 1; j++)
21                {
22                    if (orders[j].TotalPrice > orders[j + 1].TotalPrice)
23                    {
24                        Swap(orders, j, j + 1);
25                        swapped = true;
26                    }
27                }
28
29                if (!swapped)
30                {
31                    break;
32                }
33            }
34
35            0 references
36            public static Order[] BubbleSortCopy(Order[] orders)
37        }
38    }
39 }
```

Tip: Try the Plan agent to research and plan before implementing changes.

+ SortingAlgorithms

Describe what to bu...

+ ... Def...

Ln 179, Col 22 Spaces: 4 UTF-8 CRLF

```
1 namespace SortingCustomerOrders
2 {
3     47 references
4     public class Order
5     {
6         4 references
7         private int _orderId;
8         4 references
9         private string _customerName;
10        4 references
11        private double _totalPrice;
12
13        16 references
14        public Order(int orderId, string customerName, double totalPrice)
15        {
16            _orderId = orderId;
17            _customerName = customerName;
18            _totalPrice = totalPrice;
19        }
20
21        0 references
22        public int OrderId
23        {
24            get { return _orderId; }
25            set { _orderId = value; }
26        }
27
28        0 references
29        public string CustomerName
30        {
31            get { return _customerName; }
32            set { _customerName = value; }
33        }
34
35        6 references
36        public double TotalPrice
37        {
38            get { return _totalPrice; }
39        }
40    }
41 }
```

Tip: Try the Plan agent to research and plan before implementing changes.

+ Order.cs

Describe what to bu...

+ ... Def...

Ln 39, Col 2 Spaces: 4 UTF-8 CRLF

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\SortingCustomerOrders> dotnet build
Restore complete (0.5s)
SortingCustomerOrders net8.0 succeeded (0.3s) → bin\Debug\net8.0\SortingCustomerOrders.dll

Build succeeded in 1.7s
PS D:\Cognizant Hands-On\MODULE 2\SortingCustomerOrders> dotnet run
=== Sorting Customer Orders ===

1. SORTING ALGORITHMS EXPLANATION
=====

What is Sorting?
Sorting is arranging data in a particular order (ascending or descending).
In e-commerce, sorting orders by price helps prioritize high-value orders.

Different Sorting Algorithms:

1. BUBBLE SORT
- Compares adjacent elements and swaps if they're in wrong order
- Repeats until no swaps needed (array is sorted)
- Time Complexity:  $O(n^2)$  worst/average,  $O(n)$  best
- Space Complexity:  $O(1)$ 
- Simple but inefficient for large datasets
- Example: [5,3,8,1] → [3,5,8,1] → [3,5,1,8] → ...

2. INSERTION SORT
- Builds sorted array one element at a time
- Inserts each element into its correct position
- Time Complexity:  $O(n^2)$  worst/average,  $O(n)$  best
- Space Complexity:  $O(1)$ 
- Efficient for small or nearly sorted datasets
- Example: [5,3,8,1] → [3,5,8,1] → [3,5,8,1] → [1,3,5,8]

3. QUICK SORT
- Divides array into smaller sub-arrays (divide and conquer)
- Picks a pivot and partitions elements around it
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\SortingCustomerOrders> dotnet run
4. ALGORITHM COMPARISON ANALYSIS
=====

TIME COMPLEXITY COMPARISON:
=====
| Algorithm | Best Case | Average Case | Worst Case |
|-----|-----|-----|-----|
| Bubble Sort |  $O(n)$  |  $O(n^2)$  |  $O(n^2)$  |
| Quick Sort |  $O(n \log n)$  |  $O(n \log n)$  |  $O(n^2)$  |

PRACTICAL PERFORMANCE COMPARISON:
=====
For n = 1,000 orders:
Bubble Sort: ~1,000,000 comparisons
Quick Sort: ~10,000 comparisons
Quick Sort is 100x faster!

For n = 10,000 orders:
Bubble Sort: ~100,000,000 comparisons
Quick Sort: ~130,000 comparisons
Quick Sort is 770x faster!

WHY QUICK SORT IS PREFERRED OVER BUBBLE SORT:
=====

1. BETTER TIME COMPLEXITY:
- Quick Sort:  $O(n \log n)$  average
- Bubble Sort:  $O(n^2)$  average
- For large n,  $n \log n \ll n^2$ 

2. FEWER OPERATIONS:
- Quick Sort: Divides and conquers efficiently
- Bubble Sort: Compares every adjacent pair repeatedly
- Quick Sort does significantly fewer comparisons
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run

3. SORTED LIST
- O(log n) search (binary search)
- O(n) add/delete (maintain sorted order)
- Good when products need sorted display

4. COMBINED APPROACH (BEST)
- Dictionary for O(1) lookup by ID
- Secondary Dictionary for name-based search
- List for iteration and display

2. INVENTORY OPERATIONS TESTS
=====

Test 1: Adding Products
Product added: ID: 1, Name: Laptop, Qty: 50, Price: $999.99, Total: $49999.5
Product added: ID: 2, Name: Headphones, Qty: 100, Price: $149.99, Total: $14999
Product added: ID: 3, Name: Mouse, Qty: 200, Price: $29.99, Total: $5998
Product added: ID: 4, Name: Keyboard, Qty: 150, Price: $79.99, Total: $11998.5
Product added: ID: 5, Name: Monitor, Qty: 75, Price: $349.99, Total: $26249.25
Total Products: 5

Test 2: Adding Duplicate Product
Product with ID 1 already exists

Test 3: Search Product by ID
Found: ID: 3, Name: Mouse, Qty: 200, Price: $29.99, Total: $5998

Test 4: Search Products by Name
Found 1 product(s):
ID: 2, Name: Headphones, Qty: 100, Price: $149.99, Total: $14999

Test 5: Update Product
Product updated: ID: 3, Name: Wireless Mouse, Qty: 180, Price: $34.99, Total: $6298.200000000001
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run

4. TIME COMPLEXITY ANALYSIS
=====

Dictionary-Based Inventory (RECOMMENDED):
-----
| Operation | Time Complexity | Space Complexity |
|-----|-----|-----|
| Add       | O(1) average    | O(n)             |
| Update    | O(1) average    | O(n)             |
| Delete    | O(1) average    | O(n)             |
| Search(ID)| O(1) average    | O(n)             |
| Search(Name)| O(1) avg + O(k) | O(n)             |
| GetAll    | O(n)            | O(n)             |

Notes:
- O(1) average means constant time regardless of inventory size
- Worst case for Dictionary is O(n) (hash collisions)
- Space O(n) because we store n products
- Name search: O(1) to find list + O(k) to return k matches

List-Based Inventory (FOR COMPARISON):
-----
| Operation | Time Complexity | Space Complexity |
|-----|-----|-----|
| Add       | O(n)            | O(n)             |
| Update    | O(n)            | O(n)             |
| Delete    | O(n)            | O(n)             |
| Search    | O(n)            | O(n)             |
| GetAll    | O(n)            | O(n)             |

Notes:
- O(n) means time grows linearly with inventory size
- Must iterate through all products to find one
- Add is O(n) because we check for duplicates
```